

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 2 Exam Notes: Introduction to Java Programming & OOP

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Java Execution Environment	2
1.1	JDK vs. JRE vs. JVM	2
2	Java Characteristics and Features	3
3	Object-Oriented Programming (OOP) Concepts	3
3.1	The Four Pillars of OOP	4
4	Structure of a Simple Java Program	5
4.1	Standard Hello World Program	5
4.2	Key Elements Explained	5
4.3	Compiling and Running Java Programs	5
5	Data Types, Operators, and Expressions	6
5.1	Primitive vs. Reference Data Types	6
5.2	Memory Allocation Model	6
5.3	Operators and Expressions	7
6	Control Flow and Branching	7

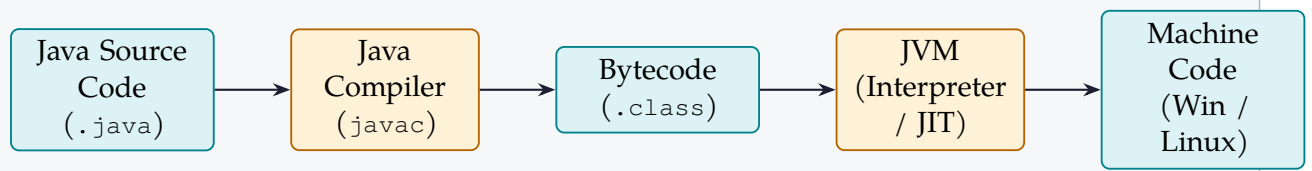
1. Java Execution Environment

Java uses a hybrid execution model combining compilation and interpretation. This ensures its core principle: **Write Once, Run Anywhere (WORA)**.

Bytecode and Platform Independence

- **Bytecode:** A highly optimized set of machine instructions designed for execution by the JVM. It is stored in a `.class` file after compilation.
- **Platform Independence:** Java source code compiles into platform-independent bytecode. However, the **JVM itself is platform-dependent** because it must translate bytecode into the host system's native machine instructions.

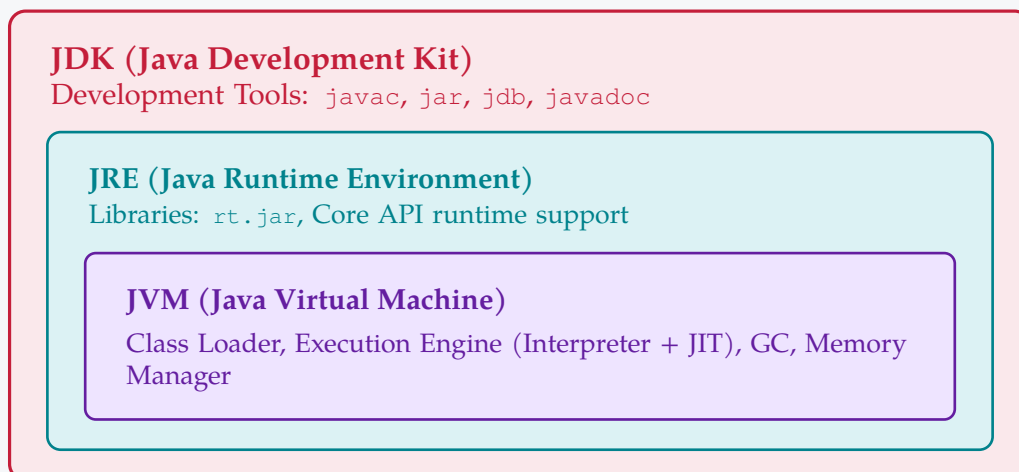
Java Compile-and-Run Execution Flow



1.1 JDK vs. JRE vs. JVM

Understanding the boundary between JVM, JRE, and JDK is crucial:

JDK, JRE, and JVM Containment Relationship



Metric	JVM (Virtual Machine)	JRE (Runtime)	JDK (Development)
Composition	Classloader, Execution Engine, Garbage Collector.	JVM + Core class libraries (<code>rt.jar</code>) + plugins.	JRE + Compiler (<code>javac</code>) + tools (<code>jar</code> , <code>jdb</code>).
Purpose	Execution of compiled bytecode.	Running Java applications.	Developing and running Java apps.
Target User	None (internal engine).	End users/clients.	Developers.

2. Java Characteristics and Features

Java's design is guided by a set of primary characteristics (often called "Java Buzzwords"):

Core Java Characteristics

1. **Simple:** Clean syntax derived from C++ but without complex features like pointers, operator overloading, or multiple inheritance.
2. **Object-Oriented:** Everything in Java is an object (except primitive types), promoting modularity and extensibility.
3. **Robust:** Emphasizes early error checking via compile-time checking, runtime exception handling, and automatic garbage collection.
4. **Secure:** Executes inside a virtual machine sandbox without direct memory manipulation or pointers, preventing buffer overflows and unauthorized system access.
5. **Portable and Architecture-Neutral:** Compiles to bytecode that can run on any device with a compatible JVM, without needing recompilation.
6. **Multithreaded:** Built-in support for concurrent execution of multiple tasks, improving resource utilization in GUI and server applications.

3. Object-Oriented Programming (OOP) Concepts

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which can contain data (in the form of fields or attributes) and code (in the form of procedures or methods). Java is a class-based, object-oriented language.

Class vs. Object

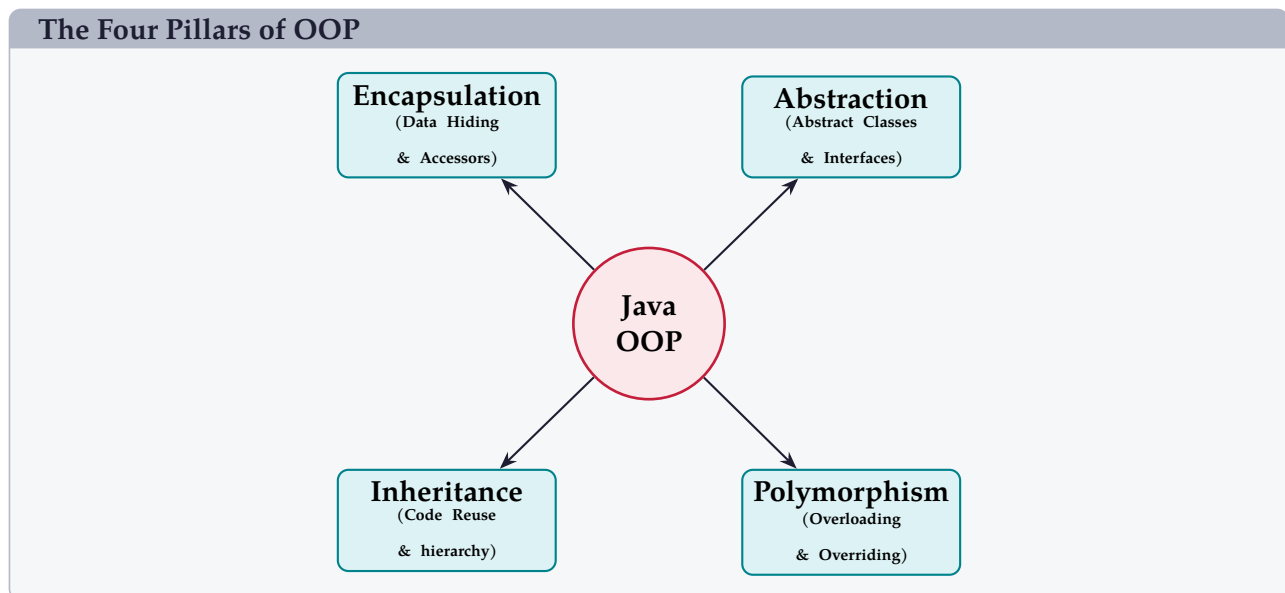
- **Class:** A logical blueprint or template used to create objects. It defines the state (attributes) and behavior (methods) that all objects of this class will share. It does not occupy memory space.

- **Object:** A physical instance of a class. It has state, behavior, and a unique identity. It occupies physical memory space on the heap.

3.1 The Four Pillars of OOP

Java fully implements the four core principles of object-oriented programming:

1. **Encapsulation (Data Hiding):** The process of wrapping data (variables) and code (methods) together as a single unit (class) while restricting direct access to the state.
 - **Implementation:** Declare instance variables as `private` and provide access via public getter and setter methods.
 - **Advantage:** Provides security, data validation, and flexibility to make variables read-only or write-only.
2. **Abstraction (Implementation Hiding):** Hiding internal implementation details and exposing only the essential features of an object.
 - **Implementation:** Achieved via `abstract` classes and `interfaces`.
 - **Advantage:** Reduces complexity and isolates the user from implementation changes.
3. **Inheritance (Code Reusability):** The mechanism by which one class (subclass) inherits the fields and methods of another class (superclass).
 - **Implementation:** Achieved using the `extends` keyword. Java supports single, multilevel, and hierarchical inheritance but does not support multiple inheritance with classes to avoid ambiguity (the "Diamond Problem").
 - **Advantage:** Promotes code reusability and establishes an "IS-A" relationship.
4. **Polymorphism (Many Forms):** The ability of a single action, method, or object to behave differently depending on the context.
 - **Compile-time (Static) Polymorphism:** Resolved during compilation. Achieved via **Method Overloading** (same method name, different parameters).
 - **Runtime (Dynamic) Polymorphism:** Resolved during runtime. Achieved via **Method Overriding** (same method name and signature in subclass) and **Dynamic Method Dispatch**.



4. Structure of a Simple Java Program

Every standard standalone Java application has an entry point class containing a main method.

4.1 Standard Hello World Program

Below is a simple Java program that prints a message to the console:

```
public class HelloWorld {  
    // Execution entry point  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

4.2 Key Elements Explained

- **public class HelloWorld:** Declares a public class named `HelloWorld`. In Java, the file name must match the name of the public class it contains (e.g., `HelloWorld.java`).
- **public static void main(String[] args):** The entry point of the program:
 - **public:** Accessible by the JVM from outside the class.
 - **static:** Can be called without creating an instance of the class.
 - **void:** Returns no value to the runtime environment.
 - **String[] args:** Array of command-line arguments passed as strings.
- **System.out.println():** Built-in output stream to print text followed by a new line to the console.

4.3 Compiling and Running Java Programs

Java programs require a two-step compile-and-run sequence:

1. **Compilation:** Run the compiler (`javac`) to convert the source code into bytecode class files:

```
javac HelloWorld.java
```

This generates a file named `HelloWorld.class` containing platform-independent bytecode.

2. **Execution:** Run the JVM interpreter (`java`) to load and execute the class file:

```
java HelloWorld
```

(Note: Do not include the `.class` extension when executing the program).

5. Data Types, Operators, and Expressions

5.1 Primitive vs. Reference Data Types

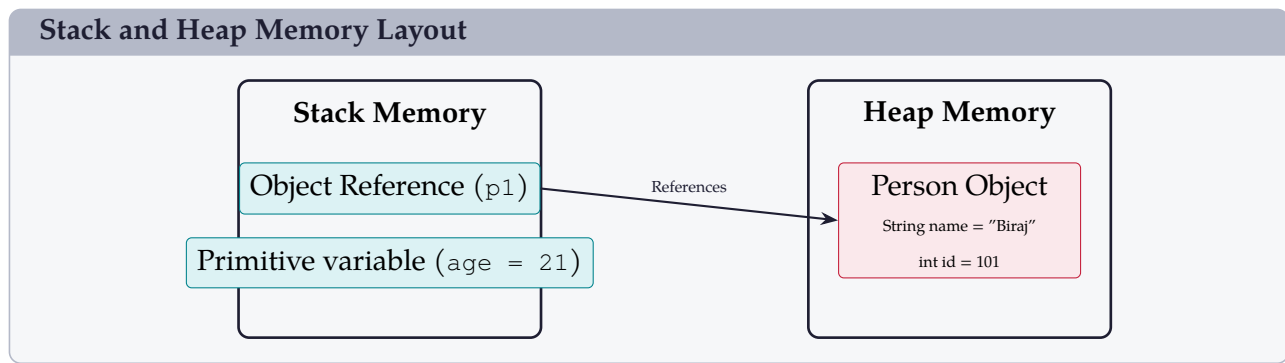
Java enforces static type checking and groups types into two primary categories:

Feature	Primitive Data Types	Reference Data Types
Allocation	Stored directly on the stack memory.	Object on Heap; address reference on Stack.
Default Value	0, 0.0, or false (never null).	null by default.
Size	Fixed sizing (e.g., int: 32-bit, double: 64-bit).	Depends on the host system architecture.
Examples	byte, short, int, long, float, double, char, boolean	Classes, interfaces, arrays, strings.

5.2 Memory Allocation Model

Java allocates memory across two distinct spaces during runtime:

- **Stack Memory:** Stores primitive variables and local references. Uses LIFO (Last In First Out) allocation.
- **Heap Memory:** Dynamic memory pool that hosts all runtime objects and instance variables.



5.3 Operators and Expressions

Java provides a comprehensive set of operators to build computational expressions:

- **Arithmetic Operators:** Binary (+, -, *, /, %) and Unary (++, --).
- **Relational Operators:** Compare operands for equality or order (==, !=, <, >, <=, >=).
- **Logical Operators:** Combine boolean expressions (&& (logical AND), || (logical OR), ! (logical NOT)). Supports short-circuit evaluation.
- **Bitwise Operators:** Perform bit-level manipulations (&, |, ^, ~, <<, >>, >>>).
- **Ternary Operator:** Conditional assignment shorthand (variable = (condition) ? valueIfTrue : valueIfFalse).

6. Control Flow and Branching

Java supports selection, iteration, and jump statements to control execution paths:

- **Selection Statements:** `if-else` blocks for boolean conditions and `switch` blocks for multi-way branching (supports byte, short, char, int, Enums, and String).
- **Iteration Statements:**
 - `for` loops for index-based loops.
 - `while` loops for condition-controlled loops (pre-test).
 - `do-while` loops for exit-controlled loops (post-test, runs at least once).
- **Jump Statements:**
 - `break`: Terminated loop or switch block immediately.
 - `continue`: Skips the rest of the current loop body and proceeds to the next iteration step.
 - **Labeled Statements:** Loops can be labeled (e.g., `outerLoop: for(...)`) to specify which loop a nested `break` or `continue` targets.

Quick Revision — Module 2

Introduction to Java Programming & OOP | OE-EC704A

Java Basic Syntax & Loop Structures

Concept	Syntax and Example Usage
Simple Program	<pre>public class Test { public static void main(String[] args) { ... } }</pre>
Data Type Sizing	byte (8-bit), char (16-bit Unicode), int (32-bit), double (64-bit)
Ternary Operator	<pre>int min = (a < b) ? a : b;</pre>
Short-Circuit logic	<pre>if (obj != null && obj.isValid()) { ... } // prevents NPE</pre>
Loop Statements	Conditional and Iterative Formats
for Loop	<pre>for(int i = 0; i < n; i++) { ... }</pre>
while Loop	<pre>while(condition) { statements; }</pre>
do-while Loop	<pre>do { statements; } while(condition);</pre>
Labeled Break	<pre>outer: for(...) { if(cond) break outer; }</pre>

One-Line Definitions

- **JVM (Java Virtual Machine):** The abstract execution engine that interprets and executes compiled Java bytecode.
- **Bytecode:** Platform-independent intermediate code representation generated by `javac`.
- **JDK:** Java Development Kit containing compiler, tools, and the JRE runtime.
- **JRE:** Java Runtime Environment providing JVM and core system class libraries.
- **Stack Memory:** LIFO memory space storing local variables and function execution frames.
- **Heap Memory:** General dynamic memory pool where all Java objects are instantiated.
- **Short-Circuiting:** Logical expression evaluation that stops as soon as the outcome is guaranteed.
- **Platform Independence:** The ability of bytecode to run unmodified on any OS running a JVM.
- **Encapsulation:** Wrapping variables and methods into a class and restricting direct access.
- **Abstraction:** Hiding execution details and exposing only the interface contract.
- **Inheritance:** Class relationship mechanism enabling child reuse of parent methods/variables.

- **Polymorphism:** Feature allowing a single entity to assume different behaviors at runtime or compile-time.

Frequently Asked / Viva Questions

Q1. Why is Java platform-independent but JVM is platform-dependent?

Java bytecode is standardized and can run on any machine. However, the JVM must be custom-tailored to the target operating system to translate that bytecode to native CPU instructions.

Q2. What is JIT Compiler in Java?

The Just-In-Time (JIT) compiler is a component of the JVM execution engine. It compiles frequently executed bytecode (hot spots) directly into native machine code at runtime to boost performance.

Q3. What is the difference between `path` and `classpath` variables?

`path` is an OS environment variable that locates shell commands like `javac` and `java`. `classpath` is a JVM-specific setting directing the classloader where to find compiled user class files.

Q4. What are the JVM memory partitions?

JVM memory is divided into: Method Area (class metadata, static variables), Heap (objects), Stack (local variables, execution frames), PC Register (current instruction address), and Native Method Stack.

Q5. Why does Java use Unicode for characters?

Java was designed for global applications. While ASCII uses 8 bits and represents 256 characters, Unicode uses 16 bits, allowing it to represent characters from all major languages.

Q6. What is the difference between prefix and postfix increment operators?

Prefix (`++i`) increments the variable first and then returns the incremented value. Postfix (`i++`) returns the original value first and then increments the variable.

Q7. Explain short-circuit logical operators.

In `A && B`, if `A` is false, the JVM skips evaluating `B` since the result is guaranteed to be false. Similarly, in `A || B`, if `A` is true, `B` is not evaluated.

Q8. Can we use a `String` in a switch-case statement?

Yes, starting from Java 7, `String` literals are supported in switch-case statements. The JVM compares strings using their `equals()` values and hash codes.

Q9. What is the difference between `break` and `continue` statements?

`break` terminates the loop completely, passing control to the statement after the loop. `continue` terminates only the current loop iteration, jumping to the loop update/condition.

Frequently Asked / Viva Questions (contd.)

Q10. Why is there no `goto` statement in Java?

Java does not support `goto` to enforce clean structured programming principles and prevent "spaghetti code." Instead, labeled `break` and `continue` statements are provided.

Q11. Explain the four pillars of OOP and their implementation in Java.

The four pillars are: Abstraction (hiding implementation details via abstract classes and interfaces), Encapsulation (data hiding using private attributes and public getters/setters), Inheritance (code reuse using subclassing with the `extends` keyword), and Polymorphism (methods behaving differently based on context via method overloading and overriding).

Q12. Why is the main method in Java declared as `public static void`?

`public` makes it accessible to the JVM from outside the class context. `static` allows the JVM to execute the method without instantiating the class first. `void` specifies that it returns no value to the operating system after execution completes.

Java Iteration Statement Comparisons

Feature	for Loop	while Loop	do-while Loop
Loop Structure	Counter-based iteration.	Condition-controlled iteration.	Exit-controlled iteration.
Condition Check	Evaluated at start.	Evaluated at start.	Evaluated at end.
Min. Runs	0 runs if condition fails.	0 runs if condition fails.	Guaranteed at least 1 run.
Best Use Case	Iterating when count is known.	Loop until event triggers.	Interactive menus.